

Multimodal Large Model + Robotic Arm Tracking

Function Description

After the program runs, input the robotic arm tracking object command through the terminal. Based on the type of object to be tracked, the large model will start an external program to control the robotic arm to track the target object.

Startup

Open the terminal and enter the following command:

```
ros2 launch largemodel largemodel_control.launch.py text_chat_mode:=True
```

Then open a second terminal and enter the following command:

```
ros2 run text_chat text_chat
```

Robotic arm tracking is divided into two types of objects:

- **Track machine code:** Track machine code x, where x is the machine code ID, values are 1-4, for example, track machine code 1.
- **Track other objects:** Track any object, for example, track the small yellow duck on the green wooden block.

Track Machine Code

Startup

In the text_chat terminal, input:

```
Track machine code 2
```

```
root@yahboom:~/# ros2 run text_chat text_chat
user input: Track machine code 2
okay@, let me think for a moment... -[INFO] [1775118028.180519381] [text_chat_node]: "action": ['apriltag_follow(2)'], "response": "I'll try to track machine code 2 again. Let me adjust my position."
user input:
```

First, the robotic arm will adjust to the tracking pose, then capture the current image and start recognizing and tracking the target machine code. If the recognized machine code is not No. 2, the robotic arm will not track and will print **"Not target apriltag"** in the terminal; if the recognized machine code is No. 2, the robotic arm will start tracking, slowly moving the machine code, and the robotic arm will follow the movement. If you want to exit tracking, input in the text_chat terminal: **Cancel tracking**.

```
user input: Cancel tracking
okay@, let me think for a moment... -[INFO] [1775118069.167061101] [text_chat_node]: "action": ['cancel_apriltag_follow(2)'], "response": "Okay, I'm canceling the tracking now."
user input: -[INFO] [1775118075.416212327] [text_chat_node]: "action": ['cancel_apriltag_follow(2)'], "response": "I attempted to track machine code 2, but it seems there's still an issue. I'll stop the tracking process now."
```

Task Planning

Call the track_pose() function to adjust the robotic arm pose to the tracking pose;

Call the apriltag_follow(2) function, where the passed parameter is 2, indicating the ID of the machine code to be tracked.

Core Code Analysis

The track_pose() function source code is located at:

LargeModel_ws/src/largemodel/largemodel/action_service.py

```
def track_pose(self): # Robotic arm tracking pose
    #Directly communicate with the bottom control board, sending the values of 6 servos of
    the robotic arm
    Arm.Arm_serial_servo_write6(90,150,12,20,90,30,1000)
    time.sleep(3.0)
    if not self.combination_mode and not self.interrupt_flag:
        self.action_status_pub("track_pose_done")
```

The apriltag_follow function source code is located at:

LargeModel_ws/src/largemodel/largemodel/action_service.py

```
def apriltag_follow(self,target_id):
    target_idf = float(target_id)
    #Open a terminal, run the tracking machine code program, passing in the parameter
    target_id, indicating the machine code ID to track
    cmd1 = f"ros2 run largemodel_arm apriltag_follow_2D --ros-args -p target_id:=
{target_idf:.1f}"
    subprocess.Popen(
        [
            "gnome-terminal",
            "--title=apriltag_follow",
            "--",
            "bash",
            "-c",
            f"{cmd1}; exec bash",
        ]
    )
```

The apriltag_follow_2D tracking machine code program source code path is:

LargeModel_ws/src/largemodel/largemodel/action_service.py

```
#Get the target tracking ID through command line parameters
self.declare_parameter('target_id', 0.0)
self.TargetID = int(self.get_parameter('target_id').get_parameter_value().double_value)
#In the image callback function callback
cur_id = tags[0].tag_id
#If the currently recognized ID is the target tracking ID
if cur_id == self.TargetID:
    #Get the center point coordinates of the machine code
    center_x, center_y = tags[0].center
    #Check if the center point coordinates are within 10 pixels of the image center
    if (abs(center_x-320) >10 or abs(center_y-240)>10) and self.XY_Track_flag==True:
        #Call the XY_track function for tracking, passing in the current center point
        coordinates
        self.XY_track(center_x,center_y)
        print("Tracking")
        print("-----")
```

```
else:
    print("Not target apriltag.")
```

Track Other Objects

Startup

In the text_chat terminal, input:

```
Track the red block in my hand
```

```
root@yahboom:~/# ros2 run text_chat text_chat
user input: Track the red block in my hand
okay@, let me think for a moment... -[INFO] [1775118700.191142704] [text_chat_node]: Decision making AI planning:1. Call 'seewhat()' to observe the environment and locate the red block in your hand.
2. Call 'track_object()' to start tracking the red block.
user input: [INFO] [1775118702.302682210] [text_chat_node]: "action": ['seewhat()'], "response": Okay, I'll track the red block in your hand. Let me take a look first!
[INFO] [1775118710.149691078] [text_chat_node]: "action": ['track(0, 190, 85, 320)'], "response": I see the red block in your hand! Now I'll start tracking it.
```

First, the robotic arm will adjust to the tracking pose, then capture a frame of the current image to find the small yellow duck in the hand, then control the robotic arm to track the small yellow duck, slowly move the position of the small yellow duck, and the robotic arm will follow the movement. If you want to exit tracking, input in the text_chat terminal: **Cancel tracking**.

Task Planning

Call the track_pose() function to adjust the robotic arm to the tracking pose;

Call the seewhat() to observe the environment and find the small yellow duck in the hand;

Call the track(x1, y1, x2, y2) function, where x1, y1, x2, y2 are the outer frame coordinates of the small yellow duck.

Core Code Analysis

The track_pose() and seewhat() functions have been analyzed before and will not be repeated here. Let's mainly look at the track function, source code located at:

LargeModel_ws/src/largemodel/largemodel/action_service.py

```
#x1,y1,x2,y2: Outer frame coordinates of the object
def track(self, x1, y1, x2, y2):
    #Start the KCF tracking program
    cmd1 = "ros2 run largemodel_arm KCF_Track_Move"
    #Start the KCF tracking positioning program
    cmd2 = "ros2 run largemodel_arm ALM_KCF_Tracker"
    subprocess.Popen(
        [
            "gnome-terminal",
            "--title=KCF_track",
            "--",
            "bash",
            "-c",
            f"{cmd1}; exec bash",
        ]
    )
    subprocess.Popen(
        [
            "gnome-terminal",
```

```

        "--title=ALM_KCF_Tracker_Node",
        "--",
        "bash",
        "-c",
        f"{cmd2}; exec bash",
    ]
)
#Publish outer frame coordinate information topic data, which will be subscribed to by
the ALM_KCF_Tracker node
x1 = int(x1)
y1 = int(y1)
x2 = int(x2)
y2 = int(y2)
self.object_position_pub.publish(Int16MultiArray(data=[x1, y1, x2, y2]))

```

The KCF_Track_Move program source code location:

LargeModel_ws/src/largemodel_arm/largemodel_arm/KCF_Track_Move.py

```

#Import library, the path of this library is
LargeModel_ws/src/largemodel_arm/largemodel_arm/Dofbot_Track.py
from largemodel_arm.Dofbot_Track import *

#Create robotic arm tracking and gripping object
self.dofbot_tracker = DofbotTrack()

#Create subscriber, subscribe to object position information topic (xy)
self.sub_grasp_status = self.create_subscription(Position,"/pos_xy",self.TrackAndGrap,100)
#Topic callback function to process received topic message data,
def TrackAndGrap(self,position):
    center_x, center_y = position.x,position.y
    #If the current object's center point coordinates are not within 8 pixels of the image
    center
    if abs(center_x-320) >8 or abs(center_y-240)>8 :
        #Call the XY_track function for tracking, passing in the current center point xy
        coordinates
        self.dofbot_tracker.XY_track(center_x,center_y)

```

```

#View the XY_track function in Dofbot_Track.py. Based on the center point coordinate values
and combined with PID parameters, calculate the values of servos 1-4
def XY_track(self,center_x,center_y):
    self.px = center_x
    self.py = center_y
    if not (self.target_servox>=180 and center_x<=320 and self.a == 1 or
self.target_servox<=0 and center_x>=320 and self.a == 1):
        if(self.a == 0):
            self.xservo_pid.SystemOutput = center_x
            if self.x_out_range == True:
                if self.target_servox<0:
                    self.target_servox = 0
                    self.xservo_pid.SetStepSignal(630)
                if self.target_servox>0:
                    self.target_servox = 180

```

```

        self.xservo_pid.SetStepSignal(10)
        self.x_out_range = False
    else:
        self.xservo_pid.SetStepSignal(320)
        self.x_out_range = False

    self.xservo_pid.SetInertiaTime(0.01, 0.1)

    target_valuex = int(1500 + self.xservo_pid.SystemOutput)

    self.target_servox = int((target_valuex - 500) / 10) - 10

    if self.target_servox > 180:
        self.x_out_range = True

    if self.target_servox < 0:
        self.x_out_range = True

    if not (self.target_servoy >= 180 and center_y <= 240 and self.b == 1 or
self.target_servoy <= 0 and center_y >= 240 and self.b == 1):
        if(self.b == 0):
            self.y servo_pid.SystemOutput = center_y

        if self.y_out_range == True:
            self.y servo_pid.SetStepSignal(450)
            self.y_out_range = False
        else:
            self.y servo_pid.SetStepSignal(240)

    self.y servo_pid.SetInertiaTime(0.01, 0.1)

    target_valuey = int(1500 + self.y servo_pid.SystemOutput)

    if target_valuey <= 1000:
        target_valuey = 1000
        self.y_out_range = True
    self.target_servoy = int((target_valuey - 500) / 10) - 55
    if self.target_servoy > 180: self.target_servoy = 180
    if self.target_servoy < 0: self.target_servoy = 0

    joint2 = 120 + self.target_servoy
    joint3 = self.target_servoy / 4.5
    joint4 = self.target_servoy / 3

    joints_0 = [float(self.target_servox/1), float(joint2), float(joint3), float(joint4),
90.0, 30.0]
    self.Arm.Arm_serial_servo_write6_array(joints_0,1000)
    self.cur_joints = joints_0

```

